

Chapter 3. The Mechanics of Modern Concurrency in Java

The knowledge of anything, since all things have causes, is not acquired or complete unless it is known by its causes.

—Ibn Sina (Avicenna), circa 1025 A.D.

Knowing how to use something is important, but understanding how it works is essential. By understanding its inner workings, we not only gain greater appreciation for it, but we can also quickly fix any problems that arise. That's why knowing how virtual threads were implemented and their internals is crucial for a well-grounded Java developer. In this chapter, we will explore how virtual threads work internally and are implemented.

We must know two concepts to understand how virtual threads are implemented. One is the `ForkJoinPool`, and the second is *continuation*.

`ForkJoinPool` is a scheduler that schedules virtual threads.

Continuation is an aspect of how virtual threads can pause their execution and then resume where they were paused. We will begin by introducing the thread pool, and then gradually get into the depth of the

`ForkJoinPool` and how it's implemented slightly differently to accommodate virtual threads.

Thread Pool

In the first chapter, we discussed the Java Executor framework, which is essentially the implementation of a thread pool. But we didn't quite explain how a thread pool works and most importantly, why we needed one.

As the name suggests, a thread pool is a group of threads that are created at the application's startup and continue running throughout the application. The strategy of creating a thread pool may differ for different kinds of pools, but the essential idea is that it will have a number of threads ready and keep them running. The thread pool will have a queue to hold the tasks. It will pick tasks from the queue and execute them as soon as threads are available. Once it finishes the execution, it will pick the next one and keep on executing tasks while the tasks are available in the

queue. If no tasks are available, it will wait until a new task is available. If all the threads are busy executing different tasks, any additional tasks are placed in the queue and wait there until the threads become available.

Why Do We Need a Thread Pool?

While creating a thread is easy (as we saw in [Chapter 1](#)), creating many threads ad hoc can be problematic. If we continue to create threads on an ad hoc basis as our demands grow, we may eventually hit the maximum number of threads allowed by the operating system, causing the application to crash—an undesirable outcome. We want to control the number of threads we can create in a running application, and a thread pool gives us this control. It also gives us a natural way of rate limiting a resource that otherwise could be overwhelmed by too many concurrent tasks. By defining how many threads are optimal for our application and setting this limit at the start, we can prevent resource exhaustion, especially when a large number of concurrent tasks might be submitted. For example, consider a web application—if a new thread is created for each incoming request, a sudden burst of requests could cause the application to crash.

In addition to providing control, a thread pool allows tasks to execute immediately without waiting for a new thread to be created, leading to faster response times. A thread pool manages threads' lifecycles. This minimizes the complexity of manually handling threads and reduces the risk of errors like thread leaks. Moreover, it gives us the sense of tasks as units of work and shifts our focus to our business logic, which we want to execute concurrently rather than dealing with the nuts and bolts of threads.

Now that we know what a thread pool is and why it's essential, let's implement a simple one from scratch to understand how it works internally.

Building a Simple Thread Pool in Java

Our `SimpleThreadPool` class will demonstrate the core concepts of a thread pool. Let's break down its implementation:

```
class Worker extends Thread {
    public Worker(ThreadGroup threadGroup, String name) {
        super(threadGroup, name);
    }
    @Override
    public void run() {
        while (running) {
            try {
                Runnable task = queue.take();
```

```

        task.run();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}
}
}

```

This `Worker` class is an inner class of the `SimpleThreadPool` class, so it has direct access to the `Queue` that we will use to hold the tasks as a form of `Runnable`. Each `Worker` class is an independent thread that continuously pulls tasks from the shared queue and executes them. The `running` flag controls the worker's lifecycles. The flag is a field of our pool, so we can gracefully shut down the pool when we want to. The idea is that as long as we have a task in the queue and the pool is running, this thread will keep trying to pull tasks from the queue. We will use a blocking queue so if no task is there, the thread will wait until it gets one.

All `Worker` threads are grouped under a `ThreadGroup`, which gives us easier management. If we want to shut down all the threads by signaling `interrupt`, we can just interrupt the group; we don't need to interrupt individually, although, under the hood, that actually happens.

Now, let's look at the implementation of our `SimpleThreadPool`:

```

import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingDeque;

public class SimpleThreadPool implements AutoCloseable {

    private final BlockingQueue<Runnable> queue;
    private final ThreadGroup threadGroup;
    private volatile boolean running = true; ❶

    public SimpleThreadPool(int poolSize, int queueSize) {
        Worker[] threads = new Worker[poolSize];
        this.queue = new LinkedBlockingDeque<>(queueSize); ❷
        this.threadGroup = new ThreadGroup("SimpleThreadPool");

        for (int i = 0; i < poolSize; i++) {
            threads[i] = new Worker(threadGroup, "Worker-" + i);
            threads[i].start(); ❸
        }
    }

    public void submit(Runnable task) {
        try {
            queue.put(task); ❹
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

```

```

    }
}

public void shutdown() {
    this.running = false;
    threadGroup.interrupt(); ❸
}

@Override
public void close() {
    while (!queue.isEmpty()) { ❹
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return; // exit gracefully
        }
    }
    shutdown();
}

class Worker extends Thread {
    public Worker(ThreadGroup threadGroup, String name) {
        super(threadGroup, name);
    }

    @Override
    public void run() {
        while (running) {
            try {
                Runnable task = queue.take();
                task.run();
            } catch (InterruptedException e) {
                // pool doesn't use interrupts for shutdown;
                // if ever interrupted, just exit
                Thread.currentThread().interrupt();
            }
        }
    }
}
}

```

The implementation reveals several important design decisions:

- ❶ The `volatile` keyword ensures all threads see changes to the `running` flag immediately, preventing workers from missing the shutdown signal.
- ❷ `LinkedBlockingDeque` provides thread-safe operations for task submission and retrieval. Its bounded capacity prevents memory exhaustion from unbounded task accumulation.
- ❸

Workers start immediately upon creation, ready to process tasks as soon as they're submitted.

- ④ The `put()` method blocks if the queue is full, providing natural backpressure when the pool is overwhelmed.
- ⑤ Interrupting the thread group signals all workers simultaneously, triggering coordinated shutdown.
- ⑥ The graceful shutdown waits for pending tasks to complete, ensuring no work is lost during pool closure.

The class has two primary methods: `submit()` and `shutdown()`. It also implements the `AutoCloseable` interface so that we can use a `try-with-resources` pattern, which is why we also have a `close` method. Its constructor initializes the thread pool with a specified number of worker threads (`poolSize`) and a queue of a given size (`queueSize`). It creates and starts the worker threads. We have used the [LinkedBlockingDeque](#) to store the tasks. It provides thread-safe operations for adding and retrieving tasks, ensuring that workers can access the queue concurrently without issues.

Using the `submit()` method, a task is added to the queue for execution by a thread from the pool. The `shutdown()` method sets the running flag to false and interrupts all threads in the pool, signaling them to stop. In the `close()` method, we gracefully shut down the thread pool. It waits a little while for the queue to empty before initiating a shutdown. That way, we can ensure all the pending tasks are proceeding.

Now that we've built our `SimpleThreadPool`, let's see it in action. Here's an example of how you might use it:

```
package ca.bazlur.modern.concurrency.c03;

public class SimpleThreadPoolDemo {

    public static void main(String[] args) throws InterruptedException {
        try (var threadPool = new SimpleThreadPool(4, 100)) {
            for (int i = 0; i < 100; i++) {
                int finalI = i;
                threadPool.submit(() -> runTask(finalI));
            }
        }

        Thread.sleep(10_000);
        System.out.println("Main thread finished");
    }

    private static void runTask(int id) {
        System.out.printf("Task %d on %s\n", id,
            Thread.currentThread().getName());
    }
}
```

```

    try {
        Thread.sleep(100); // simulate work being done
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}
}

```

In this example, we create a `SimpleThreadPool` with four worker threads and a queue capacity of one hundred. We submit a hundred tasks to the pool, and each task simply prints its ID and the name of the thread executing it. The `try-with-resources` block ensures that the `close()` method of the `SimpleThreadPool` is called when execution exits the block, allowing for graceful shutdown.

When you run this code, you'll see output like:

```

Task 1 is being executed by Worker-2
Task 0 is being executed by Worker-0
Task 2 is being executed by Worker-3
Task 3 is being executed by Worker-1

```

This demonstrates how `SimpleThreadPool` efficiently manages the execution of multiple tasks using a limited number of threads. We can experiment with different pool sizes and queue capacities to observe their impact on performance and resource utilization.

However, this example is very simplified so that we understand the basic concept. We would typically use the `ExecutorService` in a real-world application, as it provides a more robust and feature-rich thread pool implementation. For example, some thread pools have a minimum and maximum number of threads, which means that at any point in time, only a minimum number of threads will be around, but if need be, the thread pool can increase to a maximum number.

The Executor Framework

The Executor framework is designed to maximize performance while being easy to use. It provides multiple implementations that conform to a common interface, `ExecutorService`. To get instances of different thread pool implementations, we use a factory class called `Executors`, which offers various factory methods tailored for different types of thread pools.

For example, if we want a fixed-size thread pool with 10 threads, we can use the following code:

```
ExecutorService service = Executors.newFixedThreadPool(10);
```

Under the hood, the `newFixedThreadPool` method creates an instance of `ThreadPoolExecutor`, utilizing the thread count we provide along with some additional arguments. In fact, most of the other methods in the `Executors` class also use `ThreadPoolExecutor`, configuring it differently based on the arguments to achieve various behaviors.

To delve a bit deeper, let's examine the constructor of `ThreadPoolExecutor`:

```
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue,
                          ThreadFactory threadFactory,
                          RejectedExecutionHandler handler) {}
```

Then let's look at the parameters it takes:

corePoolSize

The number of threads to keep in the pool, even if they are idle, unless `allowCoreThreadTimeOut` is set.

maximumPoolSize

The maximum number of threads allowed in the pool.

keepAliveTime

When the number of threads is greater than the core, this is the maximum time that excess idle threads will wait for new tasks before terminating.

unit

The time unit for the `keepAliveTime` argument, e.g., nanoseconds, milliseconds, seconds, etc.

workQueue

The queue to use for holding tasks before they are executed. This queue will hold only the `Runnable` tasks submitted using the `execute` method.

threadFactory

The factory to use when the executor creates a new thread.

handler

The handler to use when execution is blocked because the thread bounds and queue capacities are reached.

Understanding these parameters helps us see how different configurations of thread pools are achieved through the factory methods in the `Executors` class. For instance, `newFixedThreadPool()` sets both `corePoolSize` and `maximumPoolSize` to the same value, creating a pool with a fixed number of threads that remain alive unless explicitly shut down.

Frameworks often use this class to manage task execution efficiently. For instance, the Spring Framework wraps `ThreadPool` in a `ThreadPoolTaskExecutor` to handle asynchronous operations seamlessly. Setting the appropriate pool size is crucial to achieving optimum performance.

When configuring a thread pool, two key sizes must be considered: the core pool size and the maximum pool size. The *core pool size* is the minimum number of threads always kept alive in the pool, even if idle. Maintaining a core set of threads is essential; as we know, creating new threads is expensive. By keeping these threads readily available, the system can quickly assign tasks without incurring the overhead of thread creation. These threads remain waiting when there are no tasks to execute, and they are ready to handle new tasks as they arrive.

However, having too many idle threads can waste unnecessary resources such as CPU cycles and memory. This is why setting an appropriate minimum number of threads is crucial. The *maximum pool size* defines the upper limit on the number of threads that can be active in the pool at any given time. This allows the thread pool to adjust to increased demand by scaling up when more tasks are submitted.

Determining the optimal core and maximum pool sizes is vital for performance tuning. These values can vary significantly depending on the workload and the hardware on which the application is running. Unfortunately, there is no one-size-fits-all answer. Developers often have to experiment and monitor performance metrics to find the right balance, adjusting the thread pool sizes based on empirical results. The general idea is that if the workload is CPU-bound (for example, number crunching), the number of threads should match the available CPU cores. Adding more threads can degrade performance because it increases context switching and synchronization overhead. However, if the workload is I/O-bound (e.g., database calls), having more threads than CPUs can be beneficial, as idle threads waiting on I/O can allow other tasks to be executed.

The virtual threads we discussed in the previous chapter offer a way to eliminate the complexities associated with thread pool management for the I/O-bound workload. However, while virtual threads provide a signifi-

cant advantage, the use cases for existing thread pools will not disappear entirely. Legacy systems, compatibility considerations, and specific performance requirements may still require traditional thread pools. For example, a [recent study](#) done with OpenLiberty¹ found that their own `ThreadPool` implementation performed better than virtual threads. That's why it's paramount to understand how to configure and utilize existing thread pools effectively.

Let's briefly discuss available types of thread pools in the JDK under the `Executor` framework.

FixedThreadPool

The `FixedThreadPool` creates a pool with a fixed number of threads. If all threads are busy, new tasks will wait in the queue until a thread becomes available. This approach helps maintain a consistent number of active threads, which is useful when you have a predictable workload.

Let's see an example:

```
try (ExecutorService fixedPool = Executors.newFixedThreadPool(4)) {
    for (int i = 0; i < 10; i++) {
        fixedPool.submit(() -> {
            System.out.println(Thread.currentThread().getName()
                + " is executing a task");
        });
    }
}
```

When to use: This is best suited for applications requiring a constant number of threads working concurrently, such as server applications where you need to maintain a predictable concurrency level. For instance, if you have a limited number of resources or database connections, a fixed pool can help ensure that you don't overwhelm your system.

CachedThreadPool

The `CachedThreadPool` dynamically creates new threads when needed and reuses previously created threads if they are idle. This is highly beneficial when the workload fluctuates in size, as it allows the thread pool to adjust to the demand dynamically:

```
try (ExecutorService cachedPool = Executors.newCachedThreadPool()) {
    for (int i = 0; i < 10; i++) {
        cachedPool.submit(() -> {
            System.out.println(Thread.currentThread().getName()
                + " is executing a task");
        });
    }
}
```

```

    });
}
}

```

When to use: This is ideal when you have a large number of short-lived tasks, or when tasks arrive in unpredictable bursts. It's not suitable for situations where tasks are long-running, as it could lead to too many active threads being created.

SingleThreadExecutor

The SingleThreadExecutor ensures that tasks are executed sequentially using a single thread. It's like having a single worker handling tasks in the order they are received:

```

try (var singleThreadPool = Executors.newSingleThreadExecutor()) {
    for (int i = 0; i < 5; i++) {
        singleThreadPool.submit(() -> {
            System.out.println(Thread.currentThread().getName()
                + " is executing a task");
        });
    }
}

```

When to use: Use this executor when you need tasks to run in strict sequence or when they must not run concurrently. It's often used when you're working with a shared resource that needs to be accessed by only one task at a time, ensuring no race conditions occur.

ScheduledThreadPoolExecutor

The ScheduledThreadPoolExecutor schedules tasks to run after a delay or at fixed intervals:

```

try (ScheduledExecutorService scheduledPool
    = Executors.newScheduledThreadPool(2)) {
    scheduledPool.scheduleAtFixedRate(() -> {
        System.out.println(Thread.currentThread().getName()
            + " is running a scheduled task");
    }, 0, 5, TimeUnit.SECONDS);
}

```

When to use: It is ideal for periodic tasks such as sending notifications, performing system health checks, or running maintenance tasks at regular intervals. It ensures that tasks are executed at the desired frequency without manual intervention.

WorkStealingPool

The `WorkStealingPool` creates a pool of threads that dynamically balances the workload between threads using a work-stealing algorithm. This type of pool optimizes the usage of available processors:

```
try (ExecutorService workStealingPool = Executors.newWorkStealingPool()) {
    for (int i = 0; i < 10; i++) {
        workStealingPool.submit(() -> {
            System.out.println(Thread.currentThread().getName()
                               + " is executing a task");
        });
    }
}
```

When to use: When you have a high number of small, independent tasks and want to maximize the utilization of CPU cores.

There is a special type of thread pool in Java called the `ForkJoinPool`. It is designed for divide-and-conquer tasks, where large problems can be split into smaller, independent tasks that can be processed in parallel. In fact, this `ForkJoinPool` uses a work-stealing algorithm under the hood. We'll explore the details of the `ForkJoinPool` later in this chapter.

Callable and Future: Handling Task Results

So far, we've been submitting tasks using `Runnable`, but no results have been returned. But sometimes, we need to get a result after a task is completed. This is where `Callable` and `Future` come in.

Callable

The interface looks like this:

```
public interface Callable {
    V call() throws Exception;
}
```

It is also a functional interface like `Runnable`; the only difference is that it can return a result and throw exceptions.

Let's see an example:

```
import java.util.Map;
import java.util.concurrent.*;

public class CallableExample {
```

```

static final Map<Integer, Long> cache = new ConcurrentHashMap<>(
    Map.of(0, 0L, 1, 1L) ❶
);
public static void main(String[] args) throws Exception {
    try (ExecutorService threadPool = Executors.newCachedThreadPool()) {
        Future<Long> fibonacciNumber = threadPool.submit(new Callable<Long>() {
            @Override
            public Long call() throws Exception {
                return fibonacci(50); ❸
            }
        });
    }

    private static Long fibonacci(int n) {
        if (cache.containsKey(n)) {
            return cache.get(n);
        } else {
            long result = fibonacci(n - 1) + fibonacci(n - 2);
            cache.put(n, result); ❹
            return result;
        }
    }
}

```

Let's examine the key components:

- ❶ We use a `ConcurrentHashMap` to cache Fibonacci results, making our recursive implementation efficient and thread-safe for concurrent access.
- ❷ The `submit` method accepts a `Callable<Long>` and returns a `Future<Long>`. This `Future` represents the eventual result of the asynchronous computation.
- ❸ Unlike `Runnable.run()`, the `call()` method returns a value, in this case, the 50th Fibonacci number.
- ❹ The `cache` ensures we don't recalculate values, crucial for recursive algorithms where the same values are computed multiple times.

In the preceding code, we submitted a job to calculate the 50th Fibonacci number. We used the `Callable` interface to submit the job. Since the interface is functional, we can also use a lambda expression:

```
Future fibonacciNumber = threadPool.submit(() -> fibonacci(50));
```

But look, it returns the result wrapped with another interface, `Future`. Let's find out why.

Future

Future has several methods:

```
public interface Future<V> {
    boolean cancel(boolean mayInterruptIfRunning);
    boolean isCancelled();
    boolean isDone();
    V get() throws InterruptedException, ExecutionException;
    V get(long timeout, TimeUnit unit)
        throws InterruptedException, ExecutionException, TimeoutException;
}
```

However, we usually use the `get()` and `isDone()` methods for most use cases.

When you submit a task using `Callable`, a `Future` is returned immediately, even if the task hasn't finished. The `isDone()` method checks if the task is complete, and the `get()` method retrieves the result.

NOTE

The `get()` method is a blocking operation. It will block the calling thread until the result is available.

Let's consider an example where we want to calculate multiple Fibonacci numbers:

```
public static void main(String[] args) {
    List<Future<Long>> futures = new ArrayList<>();
    List<Integer> fibonacciIndices = List.of(10, 20, 30, 40, 50);

    try (ExecutorService threadPool = Executors.newCachedThreadPool()) {
        for (int index : fibonacciIndices) {
            futures.add(threadPool.submit(() -> fibonacci(index)));
        }
        for (Future<Long> future : futures) {
            System.out.println("Fibonacci number: " + future.get());
        }
    } catch (ExecutionException | InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

In this example, multiple `Callable` tasks are submitted and executed in parallel. However, when we call `future.get()` within the loop, the main thread blocks until that specific task completes. The advantage here

is that other threads continue their work while the main thread is blocked.

The ForkJoinPool

Since Java 7, we have had a special thread pool, the `ForkJoinPool`, in addition to the generic `ThreadPoolExecutor`. Although it looks like any other pool, it has a special purpose. While `ForkJoinPool` implements the `ExecutorService` interface, it differs significantly from the traditional `ThreadPoolExecutor` in its core design and operational principles. It employs a work-stealing mechanism where each thread has its own task queue. Idle threads can steal tasks from the tails of other threads' deques. We will discuss the work-stealing algorithm in a moment. While the traditional pool deals with a shared queue where tasks are assigned to idle threads, this can lead to contention and overhead as the threads compete for tasks.

The traditional thread pool usually maintains a fixed or bounded number of threads, and thread creation and termination are less dynamic. On the other hand, `ForkJoinPool` can adaptively adjust the number of active threads based on workload and processor availability. It can also dynamically add, suspend, or resume threads to maintain efficiency.

The traditional thread pool often relies on explicit synchronization mechanisms like locks and condition variables to manage task access and thread synchronization. On the other hand, the `ForkJoinPool` minimizes the need for explicit synchronization due to the work-stealing algorithm. This allows threads to operate more autonomously, which reduces contention and overhead, thus giving us excellent performance.

Traditional thread pools don't recognize that some tasks could be dependent on other tasks. So if a subtask starts executing on a thread and then has to wait for another task, the running thread will not be able to make progress until the child task is completed by another thread. This is a problem. On the other hand, the `ForkJoinPool` is designed in such a way that tasks can be recursively decomposed into smaller subtasks, creating a tree-like structure. If a task is dependent on its child tasks, it can suspend its own execution and execute other pending tasks while waiting. Let's see an example.

Imagine we need to write a program to calculate the 20th Fibonacci number. For the sake of the demonstration, we'll need to make it multithreaded, so the assumption is we'll get it faster. We will begin with a fixed number of threads, let's say 100:

```

import java.util.Map;
import java.util.concurrent.*;

public class FibonacciNumberWithTraditionalThreadPool {
    private static final Map<Integer, Long> cache = new ConcurrentHashMap<>(
        Map.of(0, 0L, 1, 1L)
    );
    private static long getFibonacci(int i, ExecutorService pool) {
        if (cache.containsKey(i)) {
            return cache.get(i);
        }
        Future<Long> future1 = pool.submit(() -> getFibonacci(i - 1, pool)); ❶
        Future<Long> future2 = pool.submit(() -> getFibonacci(i - 2, pool)); ❷
        try {
            long l1 = future1.get(); ❸
            long l2 = future2.get();
            long result = l1 + l2;
            cache.put(i, result);
            return result;
        } catch (InterruptedException | ExecutionException e) {
            throw new RuntimeException(e);
        }
    }

    public static void main(String[] args) {
        try (var pool = Executors.newFixedThreadPool(100)) { ❹
            Future<Long> future = pool.submit(() -> getFibonacci(20, pool));
            Long l = future.get();
            System.out.println("Fibonacci number is: " + l);
        } catch (ExecutionException | InterruptedException e) {
            throw new RuntimeException(e);
        }
    }
}

```

Let's examine what we have done here:

- ❶ Each task submits two subtasks to calculate Fibonacci ($n - 1$).
- ❷ And Fibonacci ($n - 2$), creating a binary tree of tasks.
- ❸ The parent task blocks waiting for its children to complete, holding a thread hostage.
- ❹ Despite having 100 threads, the pool quickly exhausts as each thread waits for its subtasks.

The preceding Java code calculates the 20th Fibonacci number using a thread pool and caching for efficiency. It employs a recursive `getFibonacci()` function that breaks down the problem into smaller subproblems, submitting each to a thread pool. [ConcurrentHashMap](#)

acts as a cache to store previously computed results, preventing redundant calculations.

However, if we run this code, we will soon find out that it doesn't produce any result; rather, it creates a deadlock. We are limited to 100 threads here. When we call the `getFibonacci()` method, it creates two child tasks and submits them to the pool; essentially, the thread running the parent task keeps waiting until the children finish their work. Each child's task continues doing similar things, creating more tasks. While doing so, the pool runs out of threads, and they keep on waiting. Since the child tasks are in the queue and not getting executed, the other threads get stuck, essentially creating a deadlock. We can solve the deadlock by providing more threads, but we know that threads are limited.

If we generate a thread dump using the following command:

```
jcmd 36427 Thread.dump_to_file -format=json threaddump.json
```

We will see something like [Figure 3-1](#).

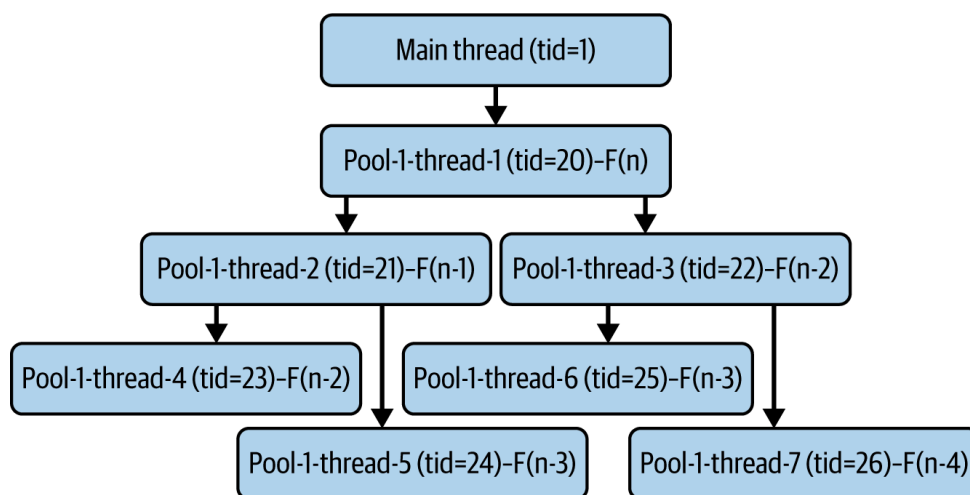


Figure 3-1. Fibonacci calculation in `ForkJoinPool`

On the other hand, if we do the same thing with a `ForkJoinPool`, it won't face a similar issue. Since `ForkJoinPool` is designed especially for the decomposition of work into multiple subproblems while waiting on the dependent tasks, it can suspend its tasks and keep working on pending tasks, and it continues working without an issue.

For instance:

```
import java.util.Map;  
import java.util.concurrent.*;  
import java.util.Map;  
import java.util.concurrent.*;
```



```

public class FibonacciNumberWithForkJoinPool {
    private static final Map<Integer, Long> cache = new ConcurrentHashMap<>(
        Map.of(0, 0L, 1, 1L)
    );

    static class FibonacciTask extends RecursiveTask<Long> { ❶
        private final int n;
        public FibonacciTask(int n) {
            this.n = n;
        }
        @Override
        protected Long compute() {
            if (cache.containsKey(n)) {
                return cache.get(n);
            }
            FibonacciTask f1 = new FibonacciTask(n - 1);
            f1.fork(); ❷
            FibonacciTask f2 = new FibonacciTask(n - 2);
            long result = f2.compute() + f1.join(); ❸
            cache.put(n, result);
            return result;
        }
    }

    public static void main(String[] args) {
        try (var pool = new ForkJoinPool()) { ❹
            Long result = pool.invoke(new FibonacciTask(20));
            System.out.println("Fibonacci number is: " + result);
        }
    }
}

```

Let's examine how `ForkJoinPool` avoids the deadlock:

- ❶ `RecursiveTask` is designed specifically for Fork/Join decomposition, providing the framework for splitting work.
- ❷ The `fork()` method asynchronously submits the subtask to the pool, placing it in the current thread's deque.
- ❸ Here's the key difference: `compute()` executes `f2` directly on the current thread (avoiding thread consumption), while `join()` waits for `f1`. If `f1` isn't complete, the thread can steal other work instead of blocking.
- ❹ `ForkJoinPool` uses a default configuration optimized for the number of available processors.

This Java code calculates the 20th Fibonacci number using a `ForkJoinPool`, which is designed for efficient parallel execution of recursive tasks. We also used a `ConcurrentHashMap` to store previously computed Fibonacci numbers to prevent redundant calculations. It uses a

`FibonacciTask` class that extends `RecursiveTask`, which allows tasks to be split and executed in parallel.

If we run the code, we will immediately get results.

Now that we have covered why `ForkJoinPool` is special, let's discuss why it is used to implement virtual threads.

NOTE

In Java, contention arises when threads need exclusive access to shared resources such as objects, variables, or I/O streams. Usually, those resources are guarded using synchronization, locks, etc. These synchronization mechanisms ensure the program's correctness, but the application's performance can degrade if there are too many contentions. For example, if threads spend more time waiting to acquire locks, it leads to longer execution times. So, if we can design our program with fewer contentions, the performance is likely to increase. In fact, the `ForkJoinPool` is designed to have lower contention, which is why it yields excellent performance.

Why ForkJoinPool for Virtual Threads?

Like the regular thread pool, the `ForkJoinPool` also comprises a predefined number of worker threads. If we don't specify the amount of parallelism (essentially the number of threads) in the constructor argument, it starts with the number of available processors in the system it runs.

The `ForkJoinPool` can be used for both CPU-bound and I/O-bound workloads; however, it is essential to understand that having more threads than available processors does not benefit CPU-intensive work. In CPU-intensive work, each thread competes for CPU time, and since we have a fixed number of cores available, having more threads than available cores means more context switch overhead. Only a number of threads equal to the available CPU cores can run simultaneously, so excess threads just wait and add no benefit. On the other hand, I/O-intensive work benefits from more threads, as some of them would be blocked while others can continue working. In the previous chapter, we discussed how virtual threads benefit I/O-intensive workloads. We will get into how `ForkJoinPool` relates to virtual threads, but first, let's explore what's happening under the hood in this pool.

Each worker thread maintains its own deque (double-ended queue), namely `workQueue`, of tasks. Tasks are typically pushed and popped from one end (following LIFO [last-in, first-out] order) by the worker thread. Another thread can steal a task from the other end (using FIFO [first-in, first-out] order). When a worker thread breaks down tasks as

part of the divide-and-conquer algorithm, it places the newly created subtasks directly into its own work queue ([Figure 3-2](#)).

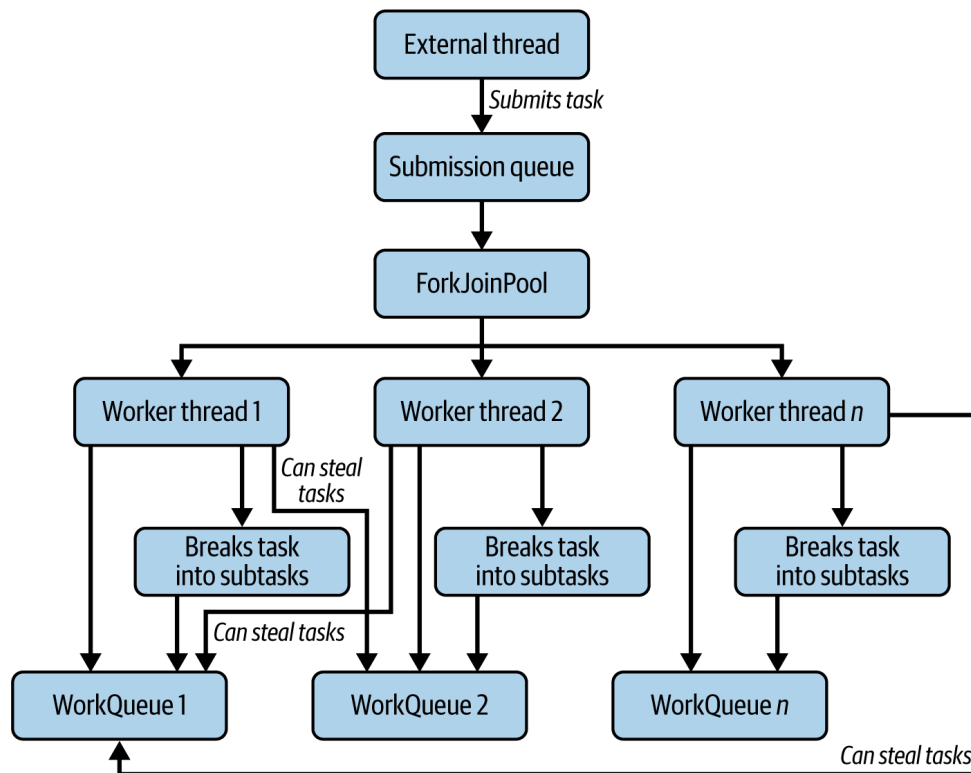


Figure 3-2. ForkJoinPool task flow—external threads submit tasks, which are distributed to worker threads that manage their own queues and can steal tasks from one another

External threads submit tasks (which are not part of the worker threads), and they enter the system through submission queues. These queues are similar to `WorkQueues` but are managed by the `ForkJoinPool` itself. Synchronization mechanisms like locks are not explicitly used; instead, atomic operations like compare-and-swap (CAS) are used to efficiently manage concurrent access.

We can use a “busy bees” metaphor to describe the worker threads. They are constantly scanning for work to keep themselves occupied. Since we have only a limited number of these worker threads, we want to ensure they’re always doing something worthwhile. When a worker runs out of tasks in its own queue, it becomes a “stealer” and attempts to steal a task from another worker’s queue. That’s where the “work-stealing” magic happens!

At first glance, one might assume that this task stealing would lead to contention when both a worker and a stealer try to access the same queue. Well, that is why, to minimize contention, two threads pick tasks from two different ends of the queue. Typically, the queue owner uses the pop method, taking from the top (LIFO), and the stealers take from the bottom (FIFO).

The “push” method adds new tasks to the head of the queue. When a worker “pops” a task, it takes the most recently added one. This makes the

queue work like a stack (LIFO). This might be another place to ask questions, as you might wonder if this is unfair to older tasks waiting in the queue.

The answer to that question is as follows—CPUs have caches that store recently accessed data. By prioritizing the newest tasks, we increase the chances that the data needed for those tasks is already in the cache. This leads to fewer “cache misses” (when the CPU has to fetch data from the main memory), thus improving performance. But what about the older tasks? Well, that’s where the stealer thread comes into the picture. It polls them from the tail of the queue.

In addition to these strategies, the `ForkJoinPool` uses clever CAS operations to manage the queue. These operations are designed to be superefficient, even when many threads are involved.

The CAS technique is used when designing concurrent algorithms. It allows for the safe and concurrent modification of a shared value without resorting to the traditional locking mechanism. Instead of acquiring a lock, a thread using CAS compares the current value of a variable with an expected value and swaps it with a new value only if the current value matches the expected one.

Consider the following example of a CAS-based atomic counter using Java's `VarHandle`. The counter is updated atomically using a CAS operation, ensuring thread safety without needing explicit locks:

```
import java.lang.invoke.MethodHandles;
import java.lang.invoke.VarHandle;

public class AtomicCounter {
    private volatile int counter = 0;
    private static final VarHandle COUNTER_HANDLE;
    static {
        try {
            COUNTER_HANDLE = MethodHandles.lookup().findVarHandle(
                AtomicCounter.class, "counter", int.class); ❶
        } catch (ReflectiveOperationException e) {
            throw new Error(e);
        }
    }

    public void increment() {
        int current;
        int next;
        do {
            current = counter; ❷
            next = current + 1;
        } while (!COUNTER_HANDLE.compareAndSet(this, current, next)); ❸
    }

    public int get() {
        return counter;
    }

    public static void main(String[] args) throws InterruptedException {
        AtomicCounter atomicCounter = new AtomicCounter();
        Thread.ofPlatform().start(() -> {
            for (int i = 0; i < 100; i++) {
                atomicCounter.increment();
            }
        });
        Thread.ofPlatform().start(() -> {
            for (int i = 0; i < 100; i++) {
                atomicCounter.increment();
            }
        });
    }
}
```

```

    });
    Thread.sleep(100);
    System.out.println("Final Counter Value: " + atomicCounter.get()); ❹
}
}

```

Let's examine how CAS ensures thread safety:

- ❶ `VarHandle` provides low-level access to variables with atomic operations. It's more performant than reflection-based approaches and offers stronger guarantees than `volatile` alone.
- ❷ We read the current value and calculate the next value. This is our "expected" state.
- ❸ The CAS operation atomically checks if the counter still equals the current value. If yes, it updates to the next and returns true. If another thread modified it, the operation returns false, and we retry.
- ❹ The final value should be exactly 200, demonstrating that no increments were lost despite concurrent access.

In this code, we have created two platform threads that increment the counter 100 times each. With the CAS operation, the value would update safely without any issues. With CAS, threads do not block each other. Instead, each thread attempts to update a shared variable atomically. If the operation fails (because another thread updated the value first), the thread retries the operation until it succeeds. This approach is non-blocking and generally more efficient, particularly under light contention, because it reduces the overhead of acquiring and releasing locks. This is precisely why `ForkJoinPool` uses CAS operations for its work-stealing queues; they keep threads productive rather than waiting.

The `ForkJoinPool` was initially designed to help speed up parallel processing by utilizing all available processor cores. This was done by employing the divide-and-conquer algorithm. The basic idea is that we can divide a big task into smaller tasks until they are simple enough to compute independently. However, one thing we have to keep in mind is that `ForkJoinPool` is the manager of the pool of threads, but it doesn't break the tasks into smaller ones itself; rather, *it is the developer's responsibility to determine how to break the tasks down*. That's where [`RecursiveTask`](#), [`RecursiveAction`](#), etc. come into the picture. The developer usually extends these classes to create their task class and add the logic into it. When a task is split into multiple child tasks, the parent task waits until its subtasks are executed; once the subtasks have finished executing, the task joins (merges) all the results into one result. While the parent task

waits for the completion of its child tasks, the worker thread can either continue working on other tasks in its queue or steal tasks from other workers if it runs out of tasks. This is uniquely different from other thread pools.

However, the use of this `ForkJoinPool` gradually expanded to many other use cases, especially with event-style tasks, which are typically designed to execute independently and do not require joining (i.e., waiting for a task's result). In such a case, a `ForkJoinPool` can be created using the appropriate constructor argument in async mode. In async mode, `ForkJoinPool` switches to FIFO, which is more suitable for tasks that do not rely on a recursive, divide-and-conquer model. It ensures that tasks are processed in the order they are submitted, which is vital for tasks representing events or external triggers.

Consider the following example:

```
import java.util.concurrent.ForkJoinPool;

public class AsyncModeExample {

    public static void main(String[] args) {
        try (ForkJoinPool forkJoinPool = new ForkJoinPool(4,
            ForkJoinPool.defaultForkJoinWorkerThreadFactory, null, true)) {
            for (int i = 0; i < 10; i++) {
                forkJoinPool.submit(new EventTask("Event " + i));
            }
        }
    }

    record EventTask(String eventName) implements Runnable {
        public void run() {
            System.out.println("Processing " + eventName
                + " in thread: " + Thread.currentThread().getName());
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            System.out.println("Completed " + eventName
                + " in thread: " + Thread.currentThread().getName());
        }
    }
}
```

In this example code, we have first created the instance of `ForkJoinPool` by passing async mode `true` to the constructor. The same can be achieved by using `Executors.newWorkStealingPool()`.

The bottom line is that `ForkJoinPool` is much more efficient and performant, which is why it has been chosen to schedule virtual threads. When using virtual threads, the `async` mode is used. The threads in the `ForkJoinPool` execute all the virtual threads and act as a scheduler.

Continuation

A *continuation* is an abstract concept that carries different meanings across fields like mathematics, computer science, and everyday language. In programming, a continuation refers to the ability of a program to save its current execution state and resume later from where it stopped.

It's similar to how a thread executes and, during a context switch, takes a snapshot of its state so it can resume later. However, in the case of continuations, this concept applies at a more granular level, even at the method level.

For instance, imagine a method with a few lines of code. Normally, when the method is called, it executes all the lines sequentially. If the method exits early and is called again, it starts from the beginning. But with a continuation, the method can “pause” at a specific point and, when called again, resume exactly from where it stopped, preserving the execution state.

Continuations have many practical uses in programming, including exception handling, non-blocking I/O operations, concurrency control, generators, etc.²

Let's explore how continuations work.

As of Java 21, continuations are available through an internal package. While using these internal APIs directly in production code is not recommended, understanding how they work can still be valuable. We can explore a sample implementation to experiment and learn more about continuations. Consider the following code:

```
import jdk.internal.vm.Continuation;
import jdk.internal.vm.ContinuationScope;
// --add-exports java.base/jdk.internal.vm=ALL-UNNAMED

public class ContinuationExample {
    public static void main(String[] args) {
        ContinuationScope scope = new ContinuationScope("main");
        Continuation continuation = new Continuation(scope, () -> {
            System.out.println("Hello from continuation");
            Continuation.yield(scope);
            System.out.println("Hello again from continuation");
        });
    }
}
```



```

        Continuation.yield(scope);
        System.out.println("Done from continuation");
    });
    System.out.println("Before starting continuation");
    continuation.run();
    System.out.println("After starting continuation");
    continuation.run();
    System.out.println("After starting continuation again");
    continuation.run();
}
}

```

In this example, we use the `Continuation` and `ContinuationScope` classes from the `jdk.internal.vm` package. These classes are not part of the public Java API and require adding special JVM arguments (`--add-exports java.base/jdk.internal.vm=ALL-UNNAMED`) to make them accessible.

WARNING

Continuation is part of the internal API; don't use it in your production code; the preceding code is only for illustration purposes. Internal APIs can change or be removed without notice, which can lead to undesirable situations.

The `ContinuationScope` is used to create a unique scope for continuation, and the `Continuation` is essentially a piece of code that can be paused and resumed.

If we run the preceding code, we will have the following output:

```

Before starting continuation
Hello from continuation
After starting continuation
Hello again from continuation
After starting continuation again
Done from continuation

```

The first line in the main prints `Before starting continuation`. When `continuation.run()` is called, the continuation starts executing. It prints `Hello from continuation` and then yields as we have called `Continuation.yield(scope)`. At this point, the continuation pauses, and control returns to main, which then prints `After starting continuation`. When `continuation.run()` is called again, it resumes from where it left off. It prints `Hello again from continuation` and yields once more. Control returns to main, which prints `After starting continuation again`. Finally, `continuation.run()` re-

sumes the continuation, and it prints `Done` from `continuation`. Since there are no more yields, the continuation is completed.

Essentially, this is how continuations work inside virtual threads. When we start executing something, if the method is being executed on the virtual thread and it starts an I/O operation, the continuation calls the `Continuation.yield(scope)` method to pause the virtual thread and remove it from being executed by a carrier thread. Later, when the I/O operation is complete, the virtual thread gets scheduled again to complete its execution.

Now, a fair question would be, how does `Continuation` do it?

We know that anything we execute gets executed by a thread, specifically a platform thread. Each thread has a stack and stack frames. When we execute code, the stack grows and moves downwards with each method invocation. Consider [Figure 3-3](#).

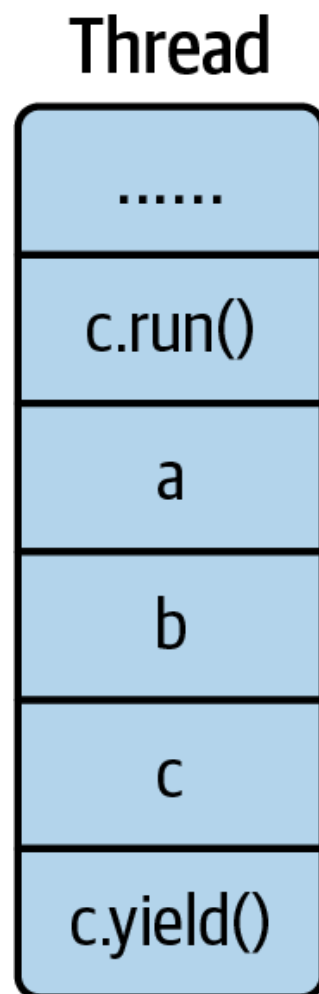


Figure 3-3. A stack and stack frame of a `Continuation.run()`

We have a platform thread that has been running, and at a certain point, it starts running the `Continuation c` by invoking `c.run()`. The `run()` method has a method `a` that calls `b`, and `b` calls `c`, and at `c`, we get `c.yield()`. That means, at this point, the `Continuation` will pause its execution and return. This `Continuation c` will be executed

by another platform thread later, but for the current thread, it needs to return. The thread will finish its current work when the `c.run()` returns. But `c.run()` hasn't really finished its work; it paused. So what it really does is take all the frames from `a` to `c.yield()` and put them aside somewhere in the `Continuation` object ([Figure 3-4](#)).

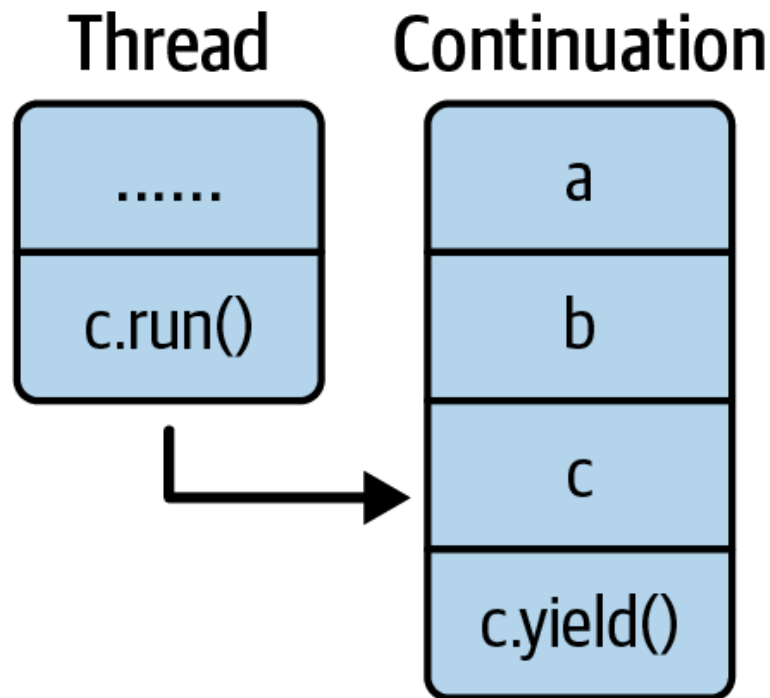


Figure 3-4. Stack moving out from the running thread after `c.yield`

The next time this `Continuation c` is mounted again, the opposite happens. All the thread frames get copied over to the thread stack, `c.yield()` returns, and execution continues where it left off. The stack then looks like [Figure 3-5](#).

Thread

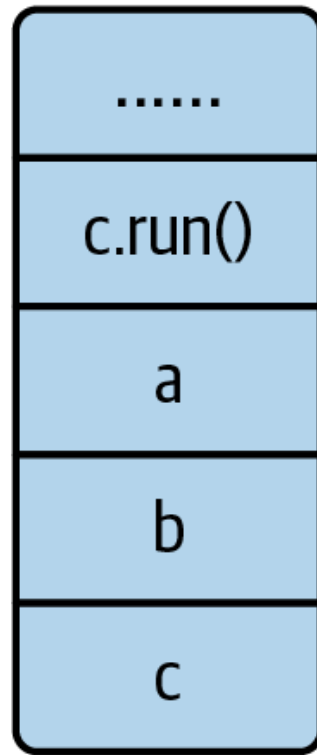


Figure 3-5. Stack frames are copied back when continuation resumes

From a high level, that's what happens; the stack frame gets copied over when a virtual thread aka continuation is unmounted and then copied back to the mounted thread again. However, this can be a costly operation, especially for deep call stacks. Each time a continuation gets suspended, the entire stack will be copied over. That's where an intelligent lazy copy mechanism is implemented in JVM.

Lazy copying is an optimization technique that avoids unnecessary copying of stack frames. What it does is, when first-time continuation is suspended, it copies over the entire stack frame to the `Continuation` object. Now, when this `Continuation` object is resumed, it doesn't copy the entire stack frame back to the thread, rather, it copies one or two frames ([Figure 3-6](#)).

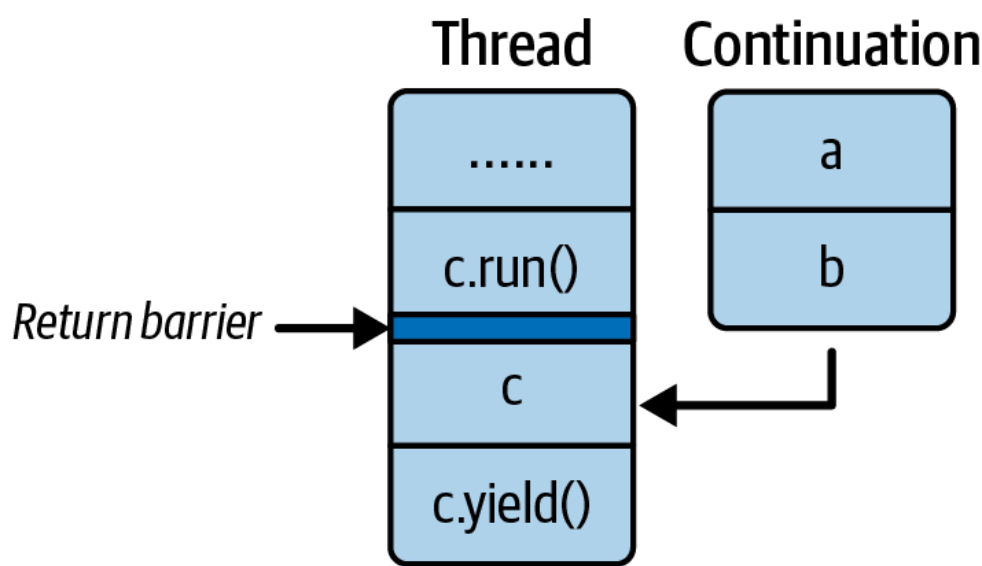


Figure 3-6. Stack copied back via lazy-copy mechanism, which doesn't copy all frames

Here, `c.yield()` returns to `c`, and `c` should return to `b`; however, `b` isn't in the stack. Instead, it employs a mechanism called *return barriers*. These are small pieces of code injected at function return points. When a function returns, the return barrier checks if the current frame needs to be copied to any continuation stacks. So, `c` doesn't return to `b` but to some VM code that will store all frames. Now it copies over `b` and may continue calling another method, say `d`. And then, if now `c.yield()` happens, only `b`, `d`, will be copied over to the Continuation object.

NOTE

I believe this provides a sufficient explanation of how Java continuation works in our day-to-day programming. However, if you want to know a bit more, I would suggest reading the source code of [Project Loom](#). There is also an [excellent talk](#) by Ron Pressler about continuations that covers pretty much everything.

Building Our Own Virtual Threads from Scratch

To illustrate the core concept of how virtual threads use continuation, we will create a simplified version of our own custom thread-like abstraction, which we'll call `NanoThread`. Using Java's Continuation API, we can simulate the behavior of virtual threads. In this section, we'll break down the code step by step, starting with the `NanoThreadScheduler`, followed by the `NanoThread` class, and finally, a simulation of file transfer operations.

The `NanoThread` class

The `NanoThread` class represents our custom virtual thread, which is built around the Continuation API. A `NanoThread` is created by passing in a `Runnable`, which is then encapsulated within a `Continuation`. Let's look at the following code:

```

import jdk.internal.vm.Continuation;
import jdk.internal.vm.ContinuationScope;
import java.util.concurrent.atomic.AtomicInteger;

public class NanoThread {
    public static final NanoThreadScheduler NANO_THREAD_SCHEDULER
        = new NanoThreadScheduler();
    private static final AtomicInteger COUNTER
        = new AtomicInteger(1);
    public static final ContinuationScope SCOPE
        = new ContinuationScope("nanoThreadScope");
    private final Continuation continuation;
    private final int nid;

    private NanoThread(Runnable runnable) {
        this.nid = COUNTER.getAndIncrement();
        this.continuation = new Continuation(SCOPE, runnable);
    }

    public static void start(Runnable runnable) {
        var nanoThread = new NanoThread(runnable);
        NANO_THREAD_SCHEDULER.schedule(nanoThread);
    }

    public void run() {
        continuation.run();
    }

    public static NanoThread currentVThread() {
        return NanoThreadScheduler.CURRENT_NANO_THREAD.get();
    }

    @Override
    public String toString() {
        return "NanoThread-" + nid + "-" + Thread.currentThread().getName()
    }
}

```

In this class, we introduce a `NanoThread` that is assigned a unique ID using `COUNTER` and manages a `Continuation`. Each `NanoThread` is initialized with a `Runnable` that encapsulates the logic to be executed.

The `run()` method is where the magic happens—it simply invokes `Continuation.run()`, allowing the thread to execute the code within the `Runnable`. If, at any point, the `NanoThread` yields its execution (using `Continuation.yield()`), it will pause and later resume from where it left off, simulating the behavior of a virtual thread.

The `start()` method initiates the `NanoThread` and hands it over to the scheduler for execution.

The NanoThread scheduler

The `NanoThreadScheduler` class plays a critical role in managing and executing `NanoThread` instances. It uses a Fork/Join Pool to handle task scheduling and a `ScheduledExecutorService` to simulate I/O-bound operations, which is a common reason threads pause and yield their execution.

Let's take a look at the scheduler:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
public class NanoThreadScheduler {
    public static final ThreadLocal<NanoThread> CURRENT_NANO_THREAD
        = new ThreadLocal<>();

    public static final ScheduledExecutorService IO_EVENT_SCHEDULER =
        Executors.newSingleThreadScheduledExecutor();
    private final ExecutorService workStealingPool =
        Executors.newWorkStealingPool(2);

    public void schedule(NanoThread nanoThread) {
        workStealingPool.submit(() -> {
            CURRENT_NANO_THREAD.set(nanoThread);
            nanoThread.run();
            CURRENT_NANO_THREAD.remove();
        });
    }
}
```

The `NanoThreadScheduler` maintains a thread-local variable `CURRENT_NANO_THREAD` to track which `NanoThread` is currently executing on a worker thread. We also define an `IO_EVENT_SCHEDULER` to simulate I/O operations that introduce delays, a critical feature for understanding the behavior of virtual threads.

The `schedule()` method submits `NanoThread` instances to the Fork/Join Pool, where the actual execution takes place. This pool operates with two worker threads, continuously checking for tasks in the queue. We can certainly add more threads to it, but for our illustration, two threads are sufficient.

Simulating I/O operations with file transfer

In our example, we simulate an I/O-bound task—a file transfer. This is where the `IO_EVENT_SCHEDULER` becomes important, as it schedules tasks with random delays to mimic real-world I/O behavior:

```

import jdk.internal.vm.Continuation;
import java.util.Random;
import java.util.concurrent.TimeUnit;
import static ca.bazlur.mcj.chap3.custom.NanoThread.NANO_THREAD_SCHEDULER;
import static ca.bazlur.mcj.chap3.custom.NanoThread.SCOPE;
import static ca.bazlur.mcj.chap3.custom.NanoThreadScheduler.CURRENT_NANO_THREAD;
import static ca.bazlur.mcj.chap3.custom.NanoThreadScheduler.IO_EVENT_SCHEDULER;

public class FileOperation {
    private final Random random = new Random();

    public void transfer(String filePath) {
        System.out.println("Start transferring file: " + filePath);
        NanoThread nanoThread = NanoThread.currentThread();
        IO_EVENT_SCHEDULER.schedule(() ->
            NANO_THREAD_SCHEDULER.schedule(nanoThread,
                random.nextInt(1000), TimeUnit.MILLISECONDS);
            CURRENT_NANO_THREAD.remove();
            Continuation.yield(SCOPE);
            System.out.println("Transfer completed for file: " + filePath);
        });
    }
}

```

The `FileOperation` class defines a `transfer()` method that simulates a file transfer. It schedules the `NanoThread` to resume execution after a random delay (simulating an I/O wait time) using the `IO_EVENT_SCHEDULER`.

In this process, the current `NanoThread` yields, giving up its execution and allowing other threads to run. When the `Continuation.yield(SCOPE)` is executed in the transfer method, the worker thread in the Fork/Join Pool assumes the work is done if the run method returns, so it goes on to execute other `NanoThread`s. Once the delay has passed, the `IO_EVENT_SCHEDULER` reschedules the `NanoThread`, allowing it to continue from where it left off.

Putting it all together

Finally, we create a demo to simulate the concurrent transfer of multiple files using our `NanoThread` system:

```

import java.time.Duration;

public class NanoThreadDemo {

    public static void main(String[] args) throws Exception {
        FileOperation fileOperation = new FileOperation();
        for (int i = 0; i < 4; i++) {
            int finalI = i;

```



```

        NanoThread.start(() -> {
            System.out.println("Transfer: "
                + "File_" + finalI + " Running in VThread: "
                + NanoThread.currentVThread());

            fileOperation.transfer("File_" + finalI);

            System.out.println("Transfer: " + "File_" + finalI
                + " Completed in VThread: " + NanoThread.currentVThread());
        });
    }

    // Let's wait for a minute to allow the
    // schedulers to run all our nano threads
    Thread.sleep(Duration.ofMinutes(1));
}
}

```

In this Demo class, we create four NanoThread s, each simulating a file transfer. We initiate the transfers concurrently, allowing them to yield and resume based on random delays.

Let's run this code using the following command:

```
java --add-exports java.base/jdk.internal.vm=ALL-UNNAMED NanoThreadDemo.java
```

We will get an output like this:

```

Transfer: File_1 Running in VThread: NanoThread-2-ForkJoinPool-1-worker-2
Transfer: File_0 Running in VThread: NanoThread-1-ForkJoinPool-1-worker-1
Start transferring file: File_1
Start transferring file: File_0
Transfer: File_2 Running in VThread: NanoThread-3-ForkJoinPool-1-worker-2
Start transferring file: File_2
Transfer: File_3 Running in VThread: NanoThread-4-ForkJoinPool-1-worker-2
Start transferring file: File_3
Transfer completed for file: File_3
Transfer: File_3 Completed in VThread: NanoThread-4-ForkJoinPool-1-worker-2
Transfer completed for file: File_0
Transfer: File_0 Completed in VThread: NanoThread-1-ForkJoinPool-1-worker-1
Transfer completed for file: File_1
Transfer: File_1 Completed in VThread: NanoThread-2-ForkJoinPool-1-worker-2
Transfer completed for file: File_2
Transfer: File_2 Completed in VThread: NanoThread-3-ForkJoinPool-1-worker-2

```

If you look closely at the preceding output, you will quickly discover that File_1 started execution by NanoThread-2-ForkJoinPool-1-worker-2 , but when it completes, it was executed by NanoThread-2-

ForkJoinPool-1-worker-1. The `NanoThread` remains the same; however, the underlying worker thread switches. This is precisely what happens in virtual threads as well. Even if a virtual thread starts on a worker thread, if it yields because of I/O, it may end up on a completely different worker when it schedules again ([Figure 3-7](#)).

Note that this implementation is only for illustration purposes.

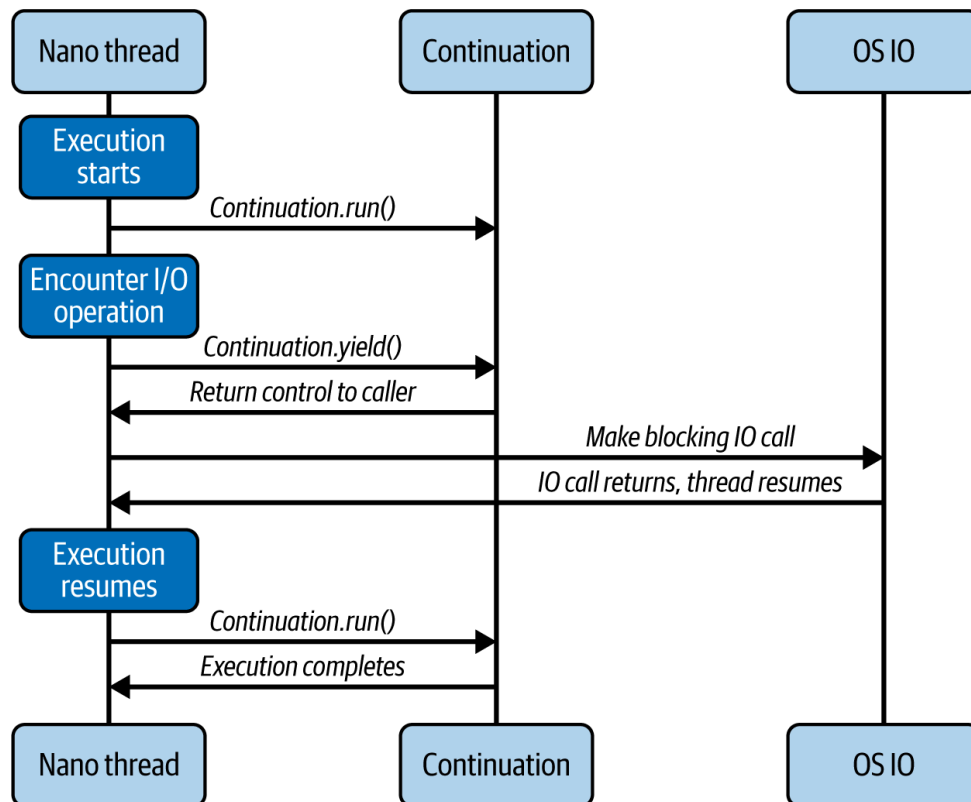


Figure 3-7. High-level design of the novel `NanoThread` to illustrate the concept

Virtual Threads and I/O Polling

In reality, when virtual threads start an I/O operation, the `Continuation` gets paused. What really happens is that the thread calls the `LockSupport.park()` [method](#). It looks like this:

```

public static void park() {
    if (Thread.currentThread().isVirtual()) {
        VirtualThreads.park();
    } else {
        U.park(false, 0L);
    }
}

```

In `VirtualThreads.park()`, after a few steps, it invokes the `yieldContinuation()` method. This effectively removes the virtual thread from the carrier threads (the real platform threads executing virtual threads).

Now, the key question is: what unparks the virtual thread when data arrives on the socket?

The JVM uses a JVM-wide read poller. At its core, this poller is a basic event loop that monitors synchronous networking operations like `read()`, `connect()`, and `accept()` when they are invoked in a virtual thread and are not immediately ready. When an I/O operation becomes ready (e.g., when data arrives on the socket), the poller gets notified and unparks the corresponding parked virtual thread. This mechanism is mirrored for write operations, with a similar write poller.

On macOS, the poller uses `kqueue`; on Linux, it uses `epoll`; and on Windows, it uses `wepoll`, which provides an epoll-like API on top of the Ancillary Function Driver for Winsock.

The poller maintains a map of file descriptors to virtual threads. When a file descriptor is registered with the poller (e.g., a socket), an entry is added to this map that associates the file descriptor with the virtual thread waiting for the I/O operation to complete. The poller's event loop, upon waking up with an event (such as data being available on a socket), uses the event's file descriptor to look up the corresponding virtual thread in the map and then unparks it, allowing it to resume execution.

If you're interested in learning more, I recommend reading the poller's [source code](#) in the OpenJDK repository. You'll see how the JVM efficiently handles I/O for virtual threads by leveraging platform-specific mechanisms like `epoll` and `kqueue` to ensure that virtual threads don't waste resources while waiting for I/O.

In Closing

Nearly all the blocking points in the JDK libraries have been rewritten to accommodate virtual threads. Most of the blocking operations now work in such a way that, when a virtual thread encounters a blocking call, it unmounts, freeing its carrier thread, the underlying OS thread, to take on new work. This approach ensures that the system can handle a large number of virtual threads without exhausting system resources.

By grasping these concepts, we will gain more confidence in working with virtual threads and become more well-rounded developers.

¹ OpenLiberty is a lightweight, cloud-native Java runtime optimized for building microservices and cloud-based applications, supporting Jakarta EE, MicroProfile, and other open standards.

2 A *generator* is a function that can yield multiple values over time, pausing its execution between yields. When the generator is called again, it resumes execution from the point where it left off, similar to how continuations work.